Detailed Solutions: TCP Congestion Avoidance

TCP Tahoe vs. TCP Reno and Textbook Problems 32–44

Computer Networks

Background and Assumptions

TCP congestion control uses Additive Increase Multiplicative Decrease (AIMD).

- **Slow start:** The congestion window approximately doubles every RTT.
- **Congestion avoidance:** The congestion window increases by approximately one MSS per RTT.
- **Timeout:** Both Tahoe and Reno set

$$\texttt{ssthresh} = \frac{\texttt{cwnd}}{2}, \qquad \texttt{cwnd} = 1 \text{ MSS}.$$

- **Triple duplicate ACK:** Tahoe resets its congestion window to one MSS. Reno performs fast retransmit and fast recovery.

Unless otherwise stated, windows are measured in MSS units. The uploaded tutorial simplifies Reno recovery by setting the post-recovery congestion window to approximately `ssthresh`.

Part A: Conceptual Questions

A1. Why does TCP use AIMD rather than a fixed window?

A fixed window cannot adapt to changing network conditions. If multiple senders transmit at fixed rates whose aggregate exceeds the bottleneck capacity, queues grow and packets are dropped.

AIMD addresses this problem through two mechanisms:

- **Additive increase:** Each sender gradually increases its congestion window to probe for available bandwidth.
- **Multiplicative decrease:** When congestion is detected, the sender reduces its congestion window, typically by half.

The increase allows senders to utilize available capacity, while the decrease reduces congestion and helps flows share the bottleneck more fairly.

A2. Why must TCP probe for available bandwidth?

TCP cannot know the available bandwidth in advance because network conditions and the number of competing flows change over time.

TCP therefore increases its congestion window gradually. It infers congestion primarily from packet loss.

A timeout indicates that an expected acknowledgement did not arrive before the timer expired. Three duplicate ACKs suggest that a segment is missing even though later segments are reaching the receiver.

A3. Why is `ssthresh` set to half the current window?

Setting the slow-start threshold to half the current congestion window provides a multiplicative reduction proportional to the load at which congestion was detected.

A fixed threshold would not adapt to different network conditions.

However, the actual sending rate need not equal `cwnd`/RTT. The sender may be application-limited, receiver-window-limited, or constrained by another network bottleneck. Consequently, the congestion window is an approximation of the amount of data the network can support.

A4. Why is a triple duplicate ACK milder than a timeout?

Three duplicate ACKs indicate that the receiver is still receiving later segments, although an earlier segment is missing. This suggests that the connection is still making progress.

A timeout indicates that the expected acknowledgement has not arrived within the timeout interval. It provides less evidence of successful progress and may indicate more severe congestion.

Duplicate ACKs also help the sender identify the likely missing segment, allowing fast retransmission.

A5. Why does Reno inflate `cwnd` during fast recovery?

Each additional duplicate ACK generally indicates that another segment has reached the receiver.

Reno increases its congestion window temporarily by one MSS for each additional duplicate ACK. This inflation helps the sender transmit data while the missing segment is being recovered.

Without inflation, the sender could leave available network capacity unused while waiting for the acknowledgement that completes recovery.

A6. Why does congestion avoidance increase the window by approximately one MSS per RTT?

A common congestion-avoidance update for each ACK is approximately

$$\Delta\texttt{cwnd} \approx \frac{\mathrm{MSS}^2}{\texttt{cwnd}}.$$

If approximately `cwnd`/MSS segments are acknowledged during one RTT, the total increase is

$$\frac{\texttt{cwnd}}{\mathrm{MSS}} \cdot \frac{\mathrm{MSS}^2}{\texttt{cwnd}} = \mathrm{MSS}.$$

Therefore, although the sender receives one ACK per segment, the increase per ACK becomes smaller as the congestion window grows. The total increase is approximately one MSS per RTT.

A7. How do delayed ACKs affect window growth?

With delayed ACKs, the receiver may send one ACK for every two segments.

If the sender increases its window by one MSS per ACK without adjustment, slow-start growth may be slower than the ideal doubling rate.

Similarly, in congestion avoidance, receiving half as many ACKs can reduce growth to approximately half an MSS per RTT.

The precise effect depends on the TCP implementation and whether it compensates for delayed acknowledgements.

A8. Why does a timeout require slow start?

A timeout indicates that the sender has lost confidence that the previous sending rate is sustainable.

TCP therefore reduces the threshold to approximately half the previous congestion window and resets the congestion window to one MSS.

The sender then probes for available bandwidth again using slow start. This cautious recovery reduces the risk of continuing to overload the network.

A9. Why do Tahoe and Reno differ when losses are frequent?

When there is no loss, both protocols increase their congestion windows in similar ways.

After a triple duplicate ACK, Tahoe resets its window to one MSS and re-enters slow start. Reno uses fast recovery and resumes congestion avoidance with a substantially larger window.

Consequently, Reno generally achieves better throughput for isolated losses detected by duplicate ACKs.

For losses detected by a timeout, both protocols reset their windows to one MSS.

A10. Why does AIMD tend toward equal sharing?

Suppose two flows share a bottleneck with congestion windows x and y.

Additive increase raises both windows at approximately the same rate. By itself, this does not eliminate the difference between the windows.

Multiplicative decrease reduces both windows proportionally. The flow with the larger window experiences a larger absolute reduction, reducing the difference between the two windows.

Repeated additive increase and multiplicative decrease can therefore move the flows toward equal sharing, assuming similar RTTs and comparable congestion signals.

Part B: TCP Tahoe vs. TCP Reno

B1. Identify slow start and congestion avoidance

According to the supplied Figure 1, both protocols initially follow approximately the same congestion-window sequence:

$$1,\ 2,\ 4,\ 8,\ 16,\ 17,\ 18,\ 19.$$

The window grows exponentially during slow start and approximately linearly during congestion avoidance.

The protocols have the same trace through round 8. They diverge after the triple duplicate ACK in round 8.

B2. Calculate the threshold after the loss in round 8

The congestion window immediately before the loss is approximately 19 MSS.

Therefore,

$$\texttt{ssthresh} = \frac{19}{2} = 9.5 \text{ MSS}.$$

For Tahoe,

$$\boxed{\texttt{ssthresh} = 9.5 \text{ MSS}, \qquad \texttt{cwnd} = 1 \text{ MSS}.}$$

Tahoe treats the triple duplicate ACK like a timeout.

For Reno, fast recovery is used. Under the simplified convention in the tutorial,

$$\boxed{\texttt{ssthresh} \approx 9.5 \text{ MSS}, \qquad \texttt{cwnd} \approx 9\text{–}9.5 \text{ MSS}.}$$

The plotted value in round 9 is approximately 9 MSS.

B3. Compare growth during rounds 9–12

Tahoe starts slow start from one MSS:

$$\texttt{cwnd}_{\text{Tahoe}} : 1,\ 2,\ 4,\ 8.$$

Reno continues congestion avoidance:

$$\texttt{cwnd}_{\text{Reno}} : 9,\ 10,\ 11,\ 12.$$

Tahoe grows exponentially, while Reno grows linearly. Reno consequently sends more data during these rounds.

B4. When does Tahoe switch to congestion avoidance?

Tahoe begins at one MSS after the loss and increases exponentially.

Its approximate window sequence is

$$1,\ 2,\ 4,\ 8,\ 9,\ 10,\ 11,\ 12.$$

Since the threshold is approximately 9.5 MSS, Tahoe switches to congestion avoidance when the window reaches or exceeds that threshold, approximately around round 14 in the supplied illustrative trace.

B5. Timeout in round 16

From Figure 1, the congestion windows immediately before the timeout are approximately

$$\texttt{cwnd}_{\text{Tahoe}} = 12 \text{ MSS}, \qquad \texttt{cwnd}_{\text{Reno}} = 16 \text{ MSS}.$$

For Tahoe,

$$\texttt{ssthresh} = \frac{12}{2} = 6 \text{ MSS}, \qquad \texttt{cwnd} = 1 \text{ MSS}.$$

For Reno,

$$\texttt{ssthresh} = \frac{16}{2} = 8 \text{ MSS}, \qquad \texttt{cwnd} = 1 \text{ MSS}.$$

Thus,

$$\boxed{\begin{aligned} &\text{Tahoe: } \texttt{ssthresh} = 6, \quad \texttt{cwnd} = 1, \\ &\text{Reno: } \texttt{ssthresh} = 8, \quad \texttt{cwnd} = 1. \end{aligned}}$$

B6. If the loss in round 16 were a triple duplicate ACK

Tahoe treats the event as a timeout:

$$\boxed{\texttt{ssthresh} = 6 \text{ MSS}, \qquad \texttt{cwnd} = 1 \text{ MSS}.}$$

Reno performs fast recovery:

$$\boxed{\texttt{ssthresh} = 8 \text{ MSS}, \qquad \texttt{cwnd} \approx 8 \text{ MSS}.}$$

Thus, in round 17, Tahoe starts at approximately one MSS, while Reno starts at approximately eight MSS under the simplified recovery convention.

B7. Total segments sent in rounds 1–16

Using the approximate window values from Figure 1:

$$\begin{aligned} \text{Tahoe: } &1, 2, 4, 8, 16, 17, 18, 19, 1, 2, 4, 8, 9, 10, 11, 12, \\ \text{Reno: } &1, 2, 4, 8, 16, 17, 18, 19, 9, 10, 11, 12, 13, 14, 15, 16. \end{aligned}$$

For Tahoe,

$$S_{\text{Tahoe}} = 1 + 2 + 4 + 8 + 16 + 17 + 18 + 19 + 1 + 2 + 4 + 8 + 9 + 10 + 11 + 12.$$

$$\boxed{S_{\text{Tahoe}} = 142 \text{ segments}.}$$

For Reno,

$$S_{\text{Reno}} = 1 + 2 + 4 + 8 + 16 + 17 + 18 + 19 + 9 + 10 + 11 + 12 + 13 + 14 + 15 + 16.$$

$$\boxed{S_{\text{Reno}} = 185 \text{ segments.}}$$

The difference is

$$185 - 142 = \boxed{43 \text{ segments}}.$$

Under this figure-based counting convention, Reno sends approximately 43 more segments by the end of round 16. This calculation uses the plotted congestion-window values and does not separately add retransmissions.

B8. Two losses in the same window

Tahoe resets its congestion window to one MSS after triple duplicate ACKs and re-enters slow start.

Basic Reno is designed to recover efficiently from a single loss within a window. If two segments are lost, the cumulative ACK may advance only to the second missing segment after the first is retransmitted.

Reno may not receive enough additional duplicate ACKs to trigger recovery of the second loss. It may therefore wait for a timeout.

This is a weakness of basic Reno with multiple losses in one window. NewReno and SACK-based TCP can handle multiple losses more effectively.

B9. Random losses unrelated to congestion

Both protocols can unnecessarily reduce their sending rates when a packet is lost due to wireless errors rather than congestion.

Tahoe suffers more after a triple duplicate ACK because it resets its congestion window to one MSS.

Reno generally performs better for isolated losses detected by duplicate ACKs because it can recover without restarting from one MSS.

If the loss causes a timeout, both protocols reset the congestion window to one MSS.

B10. Average congestion window

Suppose the congestion window oscillates between $W/2$ and W.

The average window is

$$\overline{\texttt{cwnd}} = \frac{W/2 + W}{2} = \frac{3W}{4}.$$

Therefore,

$$\boxed{\overline{\texttt{cwnd}} = 0.75W.}$$

The approximate average throughput is

$$\boxed{\text{Throughput} \approx \frac{0.75W \cdot \text{MSS}}{\text{RTT}}.}$$

B11. Two Reno flows with different RTTs

The flow with the shorter RTT generally obtains more bandwidth, assuming comparable congestion conditions.

During congestion avoidance, Reno increases its window by approximately one MSS per RTT. A flow with a shorter RTT completes more RTTs per second and therefore increases its window more quickly in wall-clock time.

Consequently, classic Reno is not inherently RTT-fair.

B12. Triple duplicate ACK in round 12

Before round 12, the initial congestion window grows approximately as

$$1, 2, 4, 8, 16, 17, 18, 19, 20, 21, 22, 23.$$

Thus, the pre-loss window is approximately 23 MSS:

$$\texttt{ssthresh} = \frac{23}{2} = 11.5 \text{ MSS}.$$

Tahoe resets its congestion window to one MSS and re-enters slow start.

Reno performs fast recovery and resumes congestion avoidance at approximately 11–11.5 MSS.

The timeout in round 16 resets each protocol to one MSS. The new threshold for each protocol depends on its own window immediately before the timeout.

The exact integer-valued thresholds depend on the convention used to interpret the illustrative graph.

Part C: Textbook Problems 32–44

P32. TCP RTT estimation

TCP estimates RTT using

$$\text{EstimatedRTT}_k = (1-\alpha)\text{EstimatedRTT}_{k-1} + \alpha\text{SampleRTT}_k.$$

Here, $\alpha = 0.1$.

(a) Four samples

Let SampleRTT_1 be the most recent sample and SampleRTT_4 the oldest of the four samples.

Expanding the recurrence gives

$$\begin{aligned}\text{EstimatedRTT}_4 = (1-\alpha)^4\text{EstimatedRTT}_0 \\ + \alpha(1-\alpha)^3\text{SampleRTT}_4 \\ + \alpha(1-\alpha)^2\text{SampleRTT}_3 \\ + \alpha(1-\alpha)\text{SampleRTT}_2 \\ + \alpha\text{SampleRTT}_1.\end{aligned}$$

Substituting $\alpha = 0.1$,

$$\begin{aligned}\text{EstimatedRTT}_4 = {} & 0.9^4\text{EstimatedRTT}_0 \\ & + 0.1(0.9)^3\text{SampleRTT}_4 \\ & + 0.1(0.9)^2\text{SampleRTT}_3 \\ & + 0.1(0.9)\text{SampleRTT}_2 \\ & + 0.1\text{SampleRTT}_1.\end{aligned}$$

(b) General formula for n samples

If samples are indexed from oldest to newest,

$$\boxed{\begin{aligned}\text{EstimatedRTT}_n = {} & (1-\alpha)^n\text{EstimatedRTT}_0 \\ & + \alpha\sum_{i=1}^{n}(1-\alpha)^{n-i}\text{SampleRTT}_i.\end{aligned}}$$

(c) Why is it called an exponential moving average?

Each older sample receives a weight containing an additional factor of $(1-\alpha)$. Its contribution therefore decreases geometrically with age.

For $0 < \alpha < 1$,

$$\lim_{n\to\infty}(1-\alpha)^n = 0.$$

The initial estimate gradually loses influence, while recent samples receive greater weights. This is why the estimator is called an exponential moving average.

P33. Why avoid RTT samples for retransmitted segments?

When a segment is retransmitted, an arriving ACK may correspond to the original transmission or the retransmission.

The sender cannot reliably determine which transmission caused the ACK, making the RTT measurement ambiguous.

Karn's algorithm therefore avoids taking RTT samples from retransmitted segments until an unambiguous sample is available.

P34. Relationship between SendBase and LastByteRcvd

`SendBase` is a sender-side variable representing the sequence number of the oldest unacknowledged byte.

`LastByteRcvd` is a receiver-side variable representing the last byte received in the relevant receiver-side description.

They are not generally equal because they describe different hosts and different stages of transmission.

The receiver's cumulative ACK progress is reflected back to the sender and can advance `SendBase`. In-flight data and missing segments can cause the variables to differ.

P35. Relationship between LastByteRcvd and y

In the standard TCP receiver-buffer description, y represents the last byte received in order, or the end of the contiguous sequence of bytes received.

If later segments arrive out of order, `LastByteRcvd` can be beyond y.

The value of y advances when the missing bytes arrive and the received data becomes contiguous.

The exact notation should be checked against the specific textbook edition assigned in the course.

P36. Why wait for three duplicate ACKs?

A single duplicate ACK can occur because of packet reordering rather than packet loss.

Retransmitting after the first duplicate ACK would therefore cause unnecessary retransmissions.

Three duplicate ACKs provide stronger evidence that a segment is missing because several later segments have arrived at the receiver.

This threshold is a practical compromise between detecting loss quickly and avoiding spurious retransmissions.

P37. Compare GBN, SR, and TCP

Assume the original transmissions are segments 1–5, segment 2 is lost, and ACK numbers indicate the next expected segment for cumulative protocols.

(a) Total transmissions and ACKs

Go-Back-N (GBN):

The sender initially transmits segments 1–5.

The receiver accepts segment 1 but discards out-of-order segments 3–5. It sends ACK 2 for segment 1 and duplicate ACK 2 for segments 3, 4, and 5.

After timeout, the sender retransmits segments 2–5.

$$\text{Data transmissions} = 5 + 4 = \boxed{9}.$$

The receiver sends one ACK for segment 1, three duplicate ACKs, and four ACKs after retransmission:

$$\text{ACKs} = 1 + 3 + 4 = \boxed{8}.$$

Selective Repeat (SR):

The sender initially transmits segments 1–5.

The receiver accepts and buffers segments 3–5 and individually acknowledges segments 1, 3, 4, and 5.

After the timer for segment 2 expires, the sender retransmits only segment 2.

$$\text{Data transmissions} = 5 + 1 = \boxed{6}.$$

The receiver sends four initial ACKs and one ACK for the retransmitted segment:

$$\text{ACKs} = 4 + 1 = \boxed{5}.$$

TCP without delayed ACK:

The sender initially transmits segments 1–5.

The receiver sends ACK 2 for segment 1 and duplicate ACK 2 for segments 3, 4, and 5.

The three duplicate ACKs trigger fast retransmission of segment 2. Once segment 2 arrives, the receiver can acknowledge all contiguous data through segment 5 using ACK 6.

$$\text{Data transmissions} = 5 + 1 = \boxed{6}.$$

Under the assumption that each received segment produces an ACK, the receiver sends four ACKs for the original arrivals and one after retransmission:

$$\text{ACKs} = \boxed{5}.$$

(b) Which protocol completes fastest?

TCP generally completes fastest because fast retransmit repairs the missing segment without waiting for a timeout.

SR retransmits only the missing segment, but under a timeout-only loss-detection assumption it must wait for the timer to expire.

GBN also waits for timeout and retransmits multiple segments, making it less efficient in this scenario.

P38. Must the actual sending rate equal `cwnd` segments per RTT?

No. The ratio `cwnd`/RTT estimates the sending rate when the congestion window is the limiting factor.

The application may not have enough data, the receiver's advertised window may be smaller, or another bottleneck may constrain transmission.

One alternative is to estimate the actual recent sending rate or flight size at the loss event and set `ssthresh` to a fraction of that estimate.

TCP uses the congestion window as a practical proxy for the amount of data the network can support.

P39. Output-rate bounds in Figures 3.46(b) and 3.46(c)

(b) Increasing the offered input rate beyond the stated threshold does not guarantee increased useful output. Retransmissions consume link capacity without delivering new data. Therefore, useful output may saturate or decline rather than exceed the model's $R/3$ bound.

(c) If each useful packet is forwarded twice on average from the router to the receiver, the repeated forwarding consumes additional link capacity. Increasing the offered load beyond the stated threshold cannot make useful output exceed the model's $R/4$ bound.

These conclusions rely on the assumptions in the textbook figures.

P40. Interpret TCP Reno Figure 3.61

Important: Figure 3.61 is not included in the uploaded tutorial document. Therefore, exact numerical answers for subparts (a)–(k) cannot be determined reliably from the supplied pages.

The solution method is as follows.

(a) Slow start corresponds to intervals where `cwnd` grows approximately exponentially.

(b) Congestion avoidance corresponds to intervals where `cwnd` grows approximately linearly.

(c) A reduction to approximately half the previous window followed by linear growth indicates triple duplicate ACK recovery in Reno. A reset to one MSS indicates a timeout.

(d) The initial `ssthresh` is the threshold at which initial exponential growth changes to linear growth.

(e) The threshold after a loss is approximately half the congestion window immediately before loss.

(f) To determine when the 70th segment is sent, add the number of segments sent in successive rounds until the cumulative total reaches 70.

(g) For a triple duplicate ACK after round 26, if the pre-loss window is W, then

$$\texttt{ssthresh} = \frac{W}{2}.$$

The post-recovery window is approximately `ssthresh` under the simplified convention.

(h) For Tahoe, a triple duplicate ACK resets `cwnd` to one MSS, after which slow start resumes.

(i) For the timeout case, sum the windows in the requested rounds using the Tahoe trace.

The textbook's Figure 3.61 is required to compute the exact intervals, thresholds, segment count, and numerical answers for all these subparts.

P41. Would AIAD converge to equal sharing?

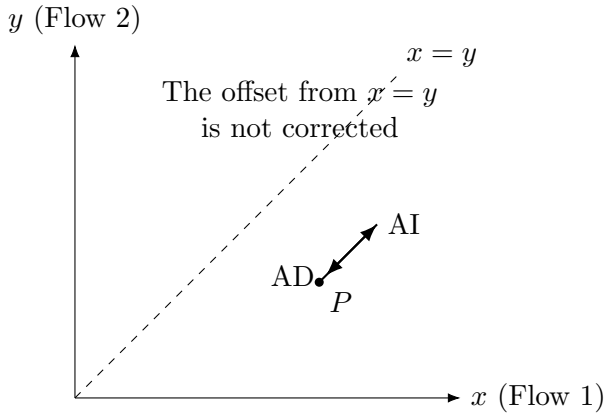
No. AIAD means Additive Increase and Additive Decrease.

Suppose two flows have windows x and y.

During additive increase, both windows increase by the same amount. Therefore, the difference $x-y$ remains constant.

During additive decrease, both windows decrease by the same fixed amount. The difference remains constant again, provided both windows remain positive.

Consequently, AIAD does not systematically correct unfair differences between flows.



P42. Why use congestion control in addition to exponential timeout backoff?

Exponential timeout backoff increases the waiting time between retransmissions after repeated timeouts. It prevents the sender from repeatedly retransmitting too aggressively when the network appears unresponsive.

However, backoff does not directly regulate how much new data can be in flight while ACKs arrive normally.

The congestion window limits outstanding data and adapts the sending rate to network congestion.

Thus, timeout backoff and window-based congestion control serve different purposes.

P43. What prevents the application from writing continuously at $S = 10R$?

The receive buffer is large enough to hold the entire file, so receiver flow control is not necessarily the limiting factor under the stated assumptions.

The sender's access link has rate R, whereas the application attempts to write at rate $10R$.

The send buffer can hold only one percent of the file. Consequently, the application can enqueue data faster than TCP can transmit it onto the network.

Eventually, the send buffer fills. A blocking socket write then blocks, while a nonblocking write may return partial progress or indicate that the buffer is full.

Therefore, the finite send buffer provides backpressure and prevents the application from continuously passing data at rate $10R$.

P44. AIMD without slow start

(a) Time for the window to grow from 6 MSS to 12 MSS

The window increases by one MSS per RTT.

The required increase is

$$12 - 6 = 6 \text{ MSS}.$$

Therefore, the time required is

$$\boxed{6\text{RTT}}.$$

(b) Average throughput through time 6RTT

Assume the initial congestion window is one MSS and the window increases by one MSS per RTT.

The windows during the first six rounds are

$$1,\ 2,\ 3,\ 4,\ 5,\ 6.$$

The total data sent is

$$1 + 2 + 3 + 4 + 5 + 6 = 21 \text{ MSS}.$$

The average throughput is

$$\begin{aligned}\text{Average throughput} &= \frac{\text{Total data sent}}{\text{Elapsed time}} \\ &= \frac{21\,\text{MSS}}{6\,\text{RTT}}.\end{aligned}$$

Hence,

$$\boxed{\text{Average throughput} = 3.5\frac{\text{MSS}}{\text{RTT}}.}$$

This answer assumes the initial window is one MSS and uses the six round-window values shown above.

Summary of Key Rules

- Slow start: approximately double `cwnd` per RTT.
- Congestion avoidance: increase `cwnd` by approximately one MSS per RTT.
- Timeout: set `ssthresh` $\approx$ `cwnd`$/2$ and reset `cwnd` to one MSS.
- Triple duplicate ACK: Tahoe resets to one MSS; Reno performs fast retransmit and fast recovery.
- Reno generally handles isolated losses better, but basic Reno can struggle with multiple losses in the same window.
- RTT estimation uses an exponentially weighted moving average.
- Retransmitted segments are excluded from RTT sampling because the acknowledgement timing is ambiguous.